```
#include <avr/io.h>
#include <util/delay.h>
#include <stdlib.h>
#include <avr/interrupt.h>

#define bitSet(reg, n)     ((reg) |= (1 << (n)))
#define bitClear(reg, n)   ((reg) &= ~(1 << (n)))
#define bitCheck(reg, n)   (((reg) >> (n)) & 1)

#define start_tc0          (TCCR0B |= 0b001)    // prescaler = 1
#define stop_tc0           (TCCR0B &= ~0b111)

#define enable_tc0_int     (bitSet(TIMSK0, TOIE0))
#define disable_tc0_int    (bitClear(TIMSK0, TOIE0))

#define pin_trigger PD4
#define pin_echo    PD7
#define pin_oc0b    PD5  // Changed from PD6 because OCR0A is now TOP

#define prescaler 1
// We now define TOP manually. 255 replicates previous behavior.
#define MY_TOP    255
//#define MY_TOP    199

float Tov;
float clock_tc0;

volatile unsigned int numOV = 0;
volatile unsigned int numOV_max_sonar = 0;

volatile uint8_t *ddr_sonar  = &DDRD;
volatile uint8_t *port_sonar = &PORTD;
volatile uint8_t *pin_sonar  = &PIND;

float vel_sound = 343.0f;

// Function Prototypes
// Debugging USART functions
void usart_debugging(void);
void usart_init(float baud_rate);
void usart_flush(void);
void usart_read_string(char *ptr_to_str);
void usart_send_byte(unsigned char data);
void usart_send_string(const char *ptr_to_str);
void usart_send_num(float num, char num_int, char num_decimal);

// void usart_init(float baud_rate);
// void usart_send_byte(unsigned char data);
// void usart_send_string(const char *ptr_to_str);
// void usart_send_num(float num, char num_int, char num_decimal);

float fun_map(float x, float x1, float x2, float y1, float y2);
float sonar(void);
void my_delay_us(unsigned long x);
void config_tc0(void);

ISR(TIMER0_OVF_vect)
{
    numOV++;
}

void config_tc0()
```



```
{
    clock_tc0 = 16.0e6 / prescaler;

    // Mode 7: Fast PWM where TOP = OCR0A
    // WGM02=1 (in TCCR0B), WGM01=1, WGM00=1
    // COM0B1=1 (Non-inverting PWM on OC0B/PD5)
    TCCR0A = (1 << COM0B1) | (1 << WGM01) | (1 << WGM00);
    TCCR0B = (1 << WGM02); // Prescaler is 0 right now, started by start_tc0 mac
ro

    OCR0A = MY_TOP;  // This sets the Period/Frequency
    OCR0B = 10;      // This sets the Duty Cycle (Brightness)

    // Tov = (TOP + 1) / clock_tc0. With TOP=255 and 16MHz, Tov = 16us
    Tov = ((float)MY_TOP + 1.0f) / clock_tc0 * 1.0e6;

    numOV_max_sonar = 12.0 / vel_sound / Tov * 1.0e6;
}

int main(void)
{
    usart_init(9600);

    bitSet(DDRD, pin_oc0b);          // PD5 as output
    bitSet(*ddr_sonar, pin_trigger);
    bitClear(*ddr_sonar, pin_echo);

    config_tc0();
    start_tc0;
    sei();

    while (1)
    {
        float D = sonar();

        usart_send_num(D, 3, 3);
        usart_send_byte(';');

        // Map distance to OCR0B (Duty Cycle) instead of OCR0A
        OCR0B = (uint8_t)fun_map(D, 10, 300, MY_TOP - 1, 10);

        usart_send_num(OCR0B, 3, 0);
        usart_send_byte('\n');

        my_delay_us(1000UL * 1000UL); // 1s delay
    }
}

float sonar(void)
{
    bitClear(*port_sonar, pin_trigger);
    my_delay_us(2);
    bitSet(*port_sonar, pin_trigger);
    my_delay_us(11);
    bitClear(*port_sonar, pin_trigger);

    while (!bitCheck(*pin_sonar, pin_echo));

    enable_tc0_int;
    numOV = 0;
    TCNT0 = 0;
```

```
    while (bitCheck(*pin_sonar, pin_echo) && numOV < numOV_max_sonar);

    disable_tc0_int;
    uint8_t tmp = TCNT0;

    float tElapse = (float)numOV * Tov + ((float)tmp / clock_tc0) * 1.0e6;
    float Dmm = (tElapse / 1.0e6) * vel_sound / 2.0f * 1000.0f;

    return Dmm;
}

void my_delay_us(unsigned long x)
{
    unsigned long numOV_max = (float)x / Tov;
    // TCNT0 now counts up to OCR0A instead of 255
    unsigned char tcnt0_max = ((float)x - (float)numOV_max * Tov) / 1.0e6 * cloc
k_tc0;

    if (numOV_max > 0)
    {
        numOV = 0;
        TCNT0 = 0;
        enable_tc0_int;
        while (numOV < numOV_max);
        disable_tc0_int;
    }

    TCNT0 = 0;
    while (TCNT0 < tcnt0_max);
}



float fun_map(float x, float x1, float x2, float y1, float y2)
{
    // 1. Clamp the input to the lower bound
    if (x <= x1) return y1;

    // 2. Clamp the input to the upper bound
    if (x >= x2) return y2;

    // 3. Perform Linear Interpolation (Lerp)
    // Formula: y = y1 + (x - x1) * (slope)
    // where slope = (y2 - y1) / (x2 - x1)
    return y1 + (x - x1) * (y2 - y1) / (x2 - x1);
}



// ... [Keep your USART helper functions exactly the same as provided] ...
void usart_init(float baud_rate){


    // Setting the baud rate register:
    //
    // Fosc, internal Crystal Arduino r3 = 16MHz
    // ubrr0 = (Fosc/prescale factor * baud rate) - 1
    // U2Xn=0, prescale factor is 2/4/2=16
    // U2Xn=1, prescale factor is 2/4=8

    float ubrr0 = ( (F_CPU / (16.0 * baud_rate)) - 1);
    int ubrr0a = (int)ubrr0;
```



```
    // Round up if neccessary to the next integer.
    if (ubrr0 - ubrr0a >= 0.5) {
        ubrr0a = ubrr0a + 1;
    }

    // Set baud rate
    UBRR0 = ubrr0a;

    // Enable transmitter and receiver, and the Interrupt Service
    // Routine on the RX buffer.
    UCSR0B = (1 << TXEN0) | (1 << RXEN0) | (1 << RXCIE0);

    // Set frame format: 8 bit data, 1 stop bit, no parity
    UCSR0C = (0 << UMSEL01) | (0 << UMSEL00) |  // Async mode
             (0 << UPM01) | (0 << UPM00) |         // No parity
             (0 << USBS0) |                        // 1 stop bit
             (1 << UCSZ01) | (1 << UCSZ00);        // 8-bit character

}


void usart_flush(void){
    char dummy;

    // Check if the UCSR0A(USART Control  and Status Register) has the
    // RXC0 flag  set.  If flag set  then there is unread  data in the
    // receive buffer.  Read what's in  the RX buffer which clears the
    // buffer.
    while (bitCheck(UCSR0A, RXC0))
        dummy = UDR0;

    ((void)dummy); // To suppress warning
}



void usart_send_byte(unsigned char data){

    // Send a byte  when the UDRE0(USART data register  empty flag) id
    // is set.  In the UCSR0A (UART contorl or status register A).  It
    // means that the USART data register is empty.

    while (!bitCheck(UCSR0A, UDRE0))
        ;
    UDR0 = data;
}




// check if const is really needed, cgpt says c++ compiler is strictor
// than c compiler.
void usart_send_string(const char *pstr){

    // note:  const means  a  read-only-string, prevents  accidentally
    // modifying  string  literals(which  causes  crashes).  String
    // literals in c  are a sequence of characters  enclosed in double
    // quotes, that is, an constant  array of characters that are null
    // terminated, ie '\0' terminated.

    // Send each byte, one at a  time, till the string terminator '\0'
```

```
    // is  sent.  '\0'  is literally  the numeric  value 0  written in
    // character form(NULL ACSII character).
    while (*pstr != '\0') {
        usart_send_byte(*pstr);
        pstr++;
    }
}



void usart_send_num(float num, char num_int, char num_decimal){

    // Send a number from MCU to PC by converting it to a string.

    // dtostrf(num,  width, precision,  string) converts  float to  an
    // ASCII string.  We add a string  terminator '\0' so the PC knows
    // when we are at the end of a string).


    char str[20];
    if (num_decimal == 0)
        dtostrf(num, num_int, num_decimal, str);
    else
        dtostrf(num, (num_int+num_decimal+1), num_decimal, str);
    str[num_int+num_decimal+1] = '\0';
    usart_send_string(str);

}
```